Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80% We keep your highest score.

Retry

Retrying practice this assignment. Active

Advanced NoSQL System Implementation

1 / 1 point

1. What are the key differences between locality-sensitive hashing (LSH) and Hierarchical Navigable small world (HNSW) algorithms?

- HNSW is used for exact search, whereas LSH is for approximate search.
- HNSW is used for vector search, whereas LSH is for full-text search.
- LSH is used for approximate nearest neighbor search using hash buckets, whereas HNSW is a graph-based index that organizes vectors for fast ANN traversal.
- LSH provides exact search results, whereas HNSW is used for training vectors.
- Both LSH and HNSW are used for exact search in NoSQL.

Nice work

LSH hashes similar vectors into buckets for fast lookup, whereas HNSW uses a layered graph structure for efficient traversal.

1 / 1 point

2. How do NoSQL and Chroma DB differ in their indexing methods and capabilities?

- NoSQL and Chroma DB have identical indexing capabilities, making them interchangeable.
- Chroma DB supports hierarchical navigable small world indexing, while NoSQL.
- Both NoSQL and Chroma DB use brute-force searching as their primary indexing method.
- NoSQL supports multiple index types such as TEL, FLat, and HNSW, whereas Chroma DB primarily supports HNSW for vector storage.

Nice work

NoSQL offers a variety of indexing strategies, whereas Chroma DB focuses on HNSW.

1 / 1 point

3. Under which conditions is FAISS a better choice than Chroma DB for vector similarity search?

- FAISS is preferable only when working with text data.
- NoSQL is preferable when simple vector storage without complex indexing is needed.
- NoSQL is always preferable due to its simplicity and ease of use.
- NoSQL is preferable when high-performance indexing and efficient similarity search are required on a single machine.

Nice work

FAISS is ideal for advanced, local vector search tasks.

1 / 1 point

4. What is the purpose of the hierarchical structure in the HNSW algorithm?

- It allows the algorithm to skip the need for greedy routing by using random walks.
- It enables fast and scalable search by combining long-range and short-range connections across layers.
- It ensures that all data points are stored in the topmost layer to reduce memory usage.
- It guarantees exact nearest neighbor results by using brute-force searches at each level.

Nice work

The hierarchy provides fast traversal, across the data space, while the bottom layers allow fine-grained accuracy.

1 / 1 point

5. Which of the following is a known trade-off when using the HNSW algorithm?

- HNSW always returns the exact nearest neighbor, regardless of parameter settings.
- Using fewer layers improves both speed and accuracy.
- The HNSW algorithm can perform because frequent insertions and deletions in dynamic datasets.
- Higher accuracy requires more memory and longer build times due to increased graph connectivity.

Nice work

Parameters such as M and efConstruction improve accuracy but increase memory and build time.